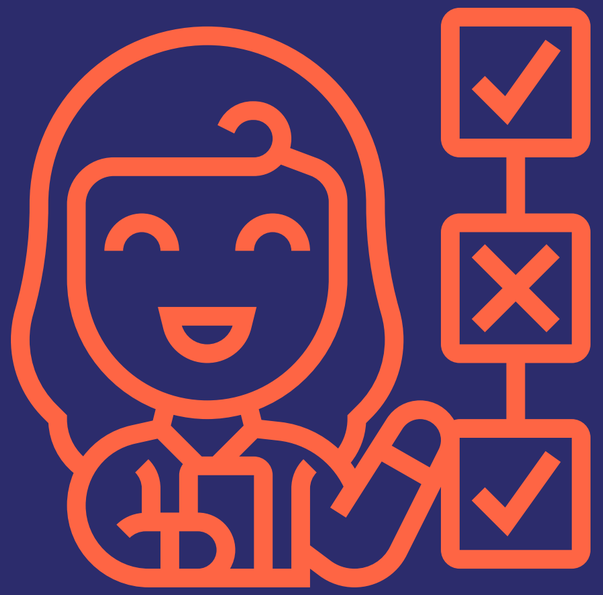
Sistema de Gestão Protocolo Sepse

Alesxandro Bryk, Eduarda Kodum, Nicholas Savagim, Gustavo Dutra

Contexto

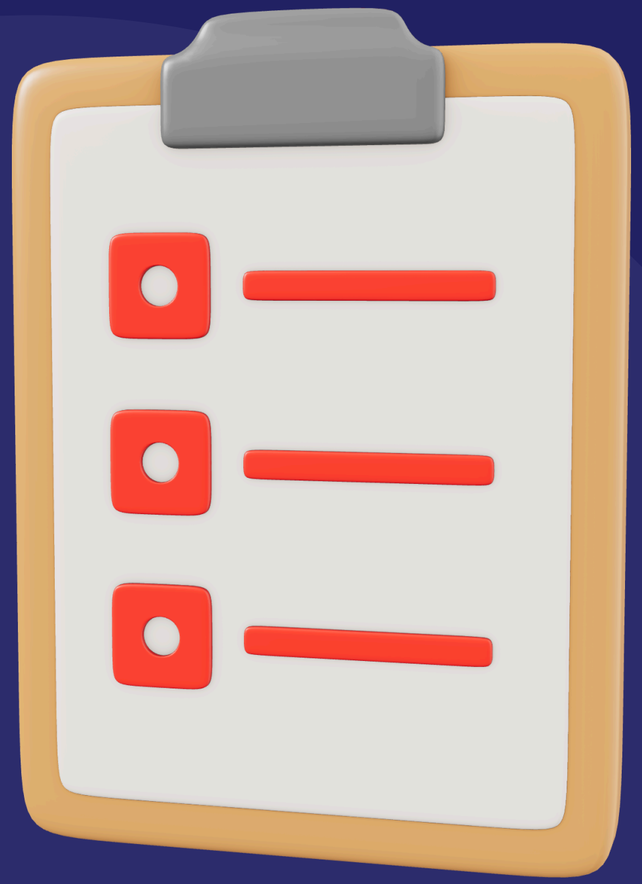


Um cenário muito comum em Hospitais e UPAs é a necessidade de seguir Protocolos rigorosos. A Sepse é uma complicação grave e uma das maiores causas de morte nesses locais, muitas vezes porque os sinais passam despercebidos ou há falha no controle do tempo.

Problema

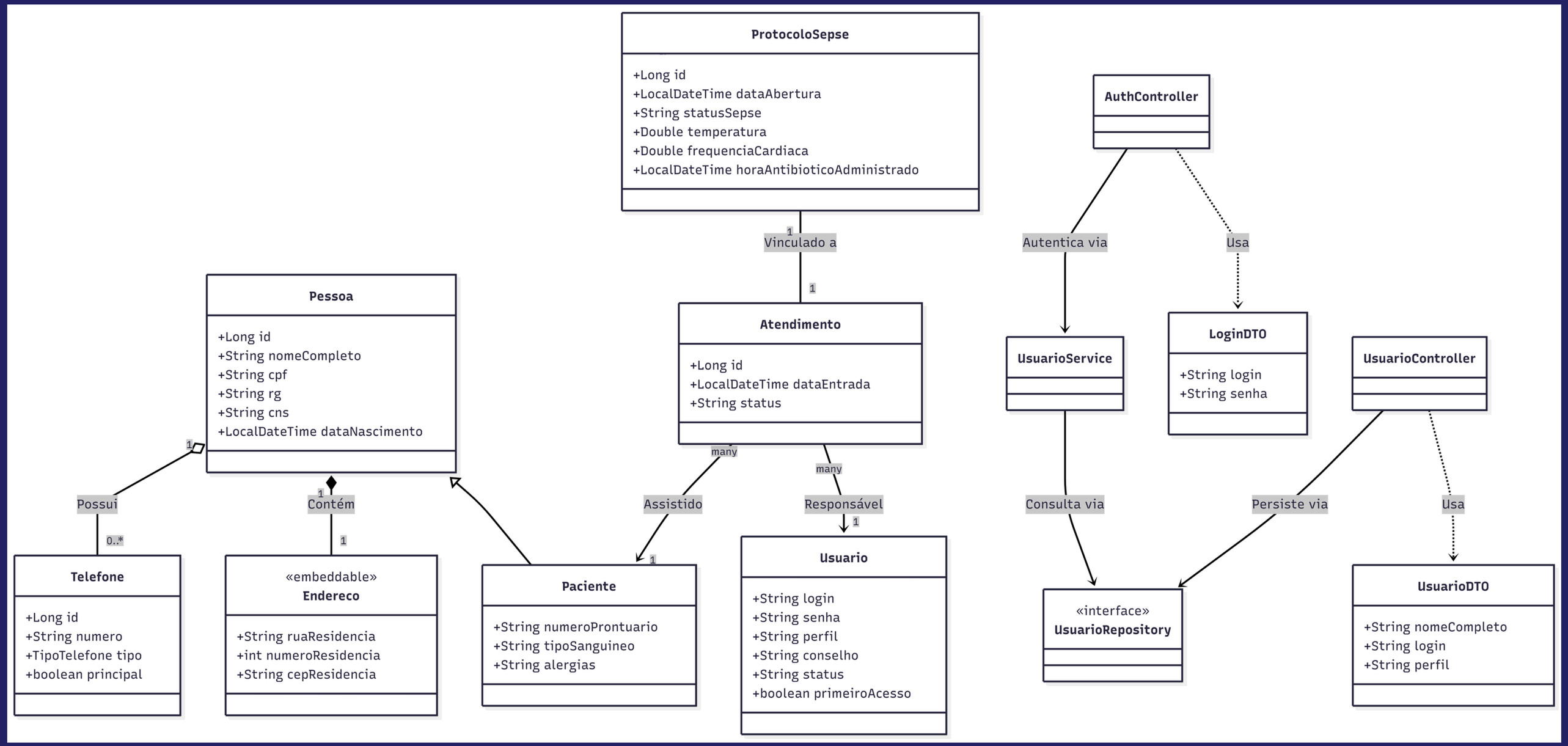
O Protocolo Sepse não tem termos clínicos complexos para o sistema, a regra de negócio dele é, basicamente, o tempo. É uma corrida contra o relógio: quando o paciente apresenta os sinais, abre-se uma 'janela de ouro' de 1 hora. Nosso sistema entra exatamente aí

Objetivo



O objetivo do nosso projeto é trazer um **controle digitalizado** para a equipe de saúde. Ele **cronometra, alerta e cria um checklist** obrigatório para os médicos e enfermeiros. Assim, garantimos que ninguém esqueça de ministrar o antibiótico ou pedir os exames dentro desse prazo crítico, evitando a deterioração clínica do paciente.

UML

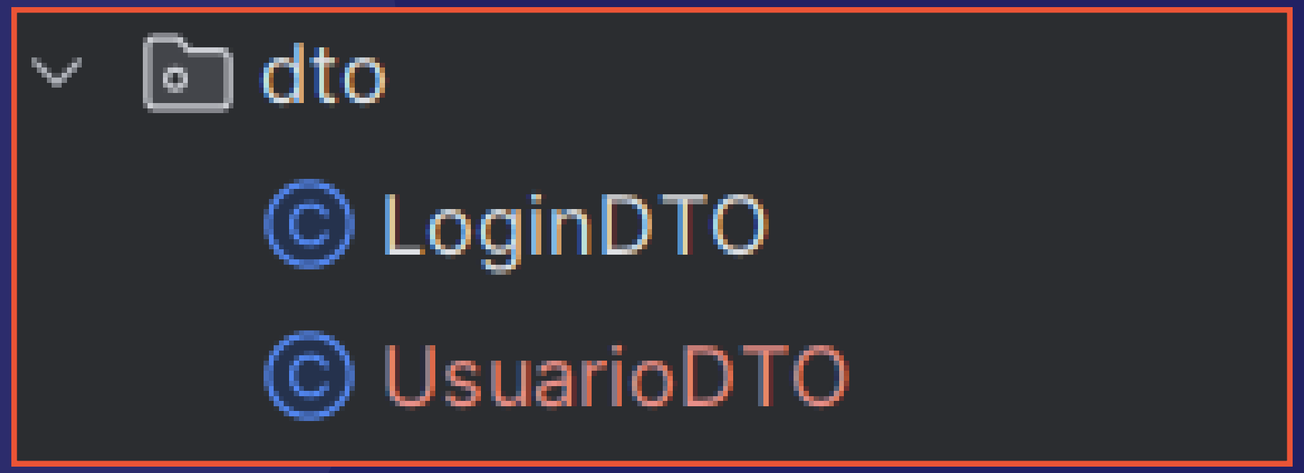

ProtocoloSepse
+Long id
+LocalDateTime dataAbertura
+String statusSepse
+Double temperatura
+Double frequenciaCardiaca
+LocalDateTime horaAntibioticoAdministrado
1
Vinculado a
1
Atendimento
+Long id
+LocalDateTime dataEntrada
+String status
many
many
Assistido
Responsável
1
1
Pessoa
+Long id
+String nomeCompleto
+String cpf
+String rg
+String cns
+LocalDateTime dataNascimento
1
Possui
0..*
1
Contém
1
Telefone
+Long id
+String numero
+TipoTelefone tipo
+boolean principal
«embeddable»
Endereco
+String ruaResidencia
+int numeroResidencia
+String cepResidencia
Paciente
+String numeroProntuario
+String tipoSanguineo
+String alergias
Usuario
+String login
+String senha
+String perfil
+String conselho
+String status
+boolean primeiroAcesso
AuthController
Autentica via
Usa
UsuarioService
LoginDTO
+String login
+String senha
UsuarioController
Consulta via
Persiste via
Usa
«interface»
UsuarioRepository
UsuarioDTO
+String nomeCompleto
+String login
+String perfil

Conceitos Aplicados

- Entidades (Entities): Onde aplicamos a Herança. Usuario e Paciente são especializações. Note que usamos @Inheritance(strategy = InheritanceType.JOINED), o que significa que no banco MySQL temos tabelas separadas, mas no Java elas são uma hierarquia sólida.

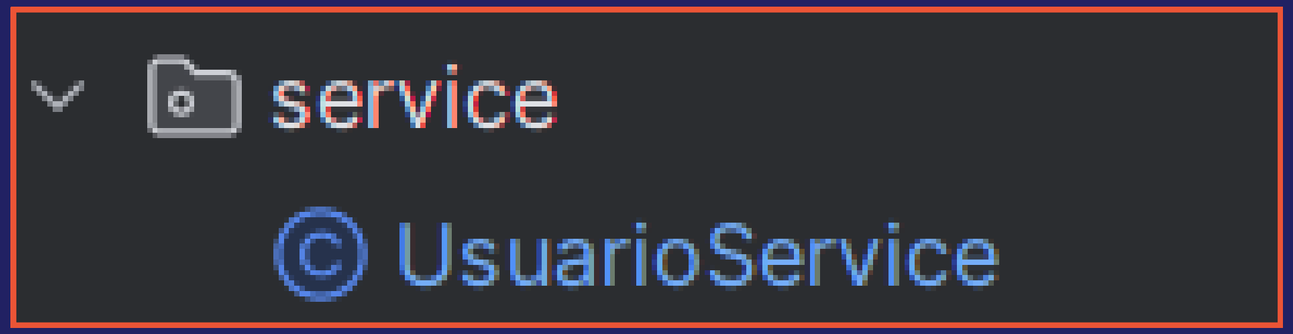
Conceitos Aplicados

- DTOs (Data Transfer Objects): O UsuarioDTO e LoginDTO servem para proteger o sistema. Sendo assim, não expomos a nossa entidade de banco de dados diretamente para a internet. O DTO filtra o que o JavaScript recebe.



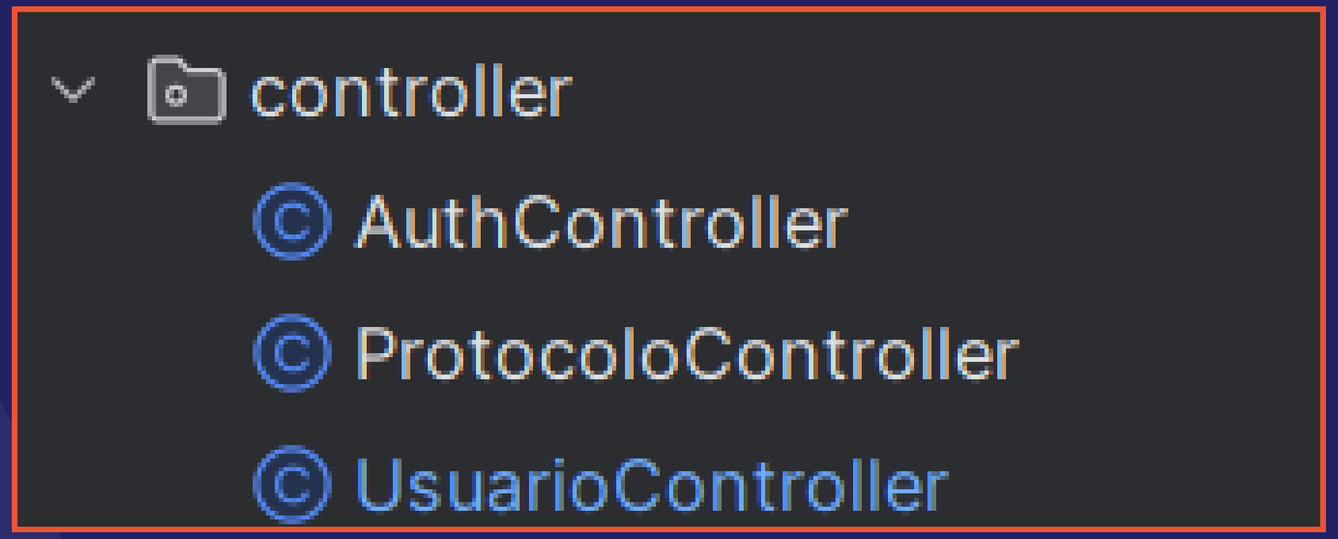
Conceitos Aplicados

- O Controller não sabe como validar uma senha; ele apenas pede ao Service para autenticar(). O Service verifica o status (se o usuário está INATIVO) antes de permitir o acesso.



Conceitos Aplicados

- O UsuarioController gerencia desde o cadastro até o reset de senha, usando Coleções (List<UsuarioDTO>) e Streams do Java para converter dados.



Encapsulamento

Como lidamos com tempos críticos, atributos como horaAberturaProtocolo são private. Eles só podem ser alterados por métodos específicos que validam a regra de negócio, nunca diretamente.

```java
@JoinColumn(name = "atendimento_id")
private Atendimento atendimento;

private LocalDateTime dataAbertura;
private String statusSepse;
private String responsavelAbertura;

private Double frequenciaCardiaca;
private Double frequenciaRespiratoria;
private Double temperatura;
private Double pressaoSistolica;
private Double pressaoDialostica;
private Double saturacao;
private LocalDateTime horaLactatoSolicitado;
private LocalDateTime horaLactadoColetado;
private LocalDateTime horaAntibioticoPrescrito;
private LocalDateTime horaAntibioticoAdministrado;
```

Herança

Utilizamos a hierarquia de classes. A classe Pessoa é a nossa generalização (contém Nome, CPF, Telefone), enquanto Usuario e Paciente são especializações que herdam esses atributos automaticamente, evitando a repetição de código

```java
package protocolo.demo.model.entity;

import ...

@Entity  16 usages  xLuanKyrb *
@Table(name = "usuario")
@Data
@EqualsAndHashCode(callSuper = true)
@PrimaryKeyJoinColumn(name = "pessoa_id")
public class Usuario extends Pessoa {

    @Column(unique = true, nullable = false)
    private String login;

    @Column(nullable = false)
    private String senha;

    private String perfil;
    private String conselho;
    private String status = "ATIVO";
    private boolean primeiroAcesso = true;
}
```

Interface

Serve para definir contratos de persistência. O UsuarioRepository assina um contrato com o Spring Data JPA. Isso isola nossa camada de serviço da implementação real do banco de dados, seguindo o princípio de abstração da OO.

```
package protocolo.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import protocolo.demo.model.entity.Usuario;

import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  4 usages  xLuanKyrb
    Optional<Usuario> findByLogin(String login);  2 usages  xLuanKyrb
}
```

Coleções

Trabalhamos com o List para gerenciar múltiplos objetos na memória. É através dessa coleção que conseguimos enviar para o nosso Frontend a lista de todos os operadores ativos que vocês viram na tela do sistema.

```
@RestController  no usages  xLuanKyrb *
@RequestMapping("/api/usuarios")
@CrossOrigin(origins = "*")
public class UsuarioController {

    @Autowired  11 usages
    private UsuarioRepository repository;

    // 1 - Listar Usuário
    @GetMapping  no usages  new *
    public List<UsuarioDTO> listarTodos() {
        return repository.findAll().stream()  Stream<Usuario>
                .map(this::converterParaDTO)  Stream<UsuarioDTO>
                .collect(Collectors.toList());
    }
```

Sobrecarga

Utilizamos as anotações @NoArgsConstructor e @AllArgsConstructor do Lombok na nossa classe Usuario.

```
@AllArgsConstructor  16 usages  xLuanKyrb *
@NoArgsConstructor
@Entity
@Table(name = "usuario")
@Data
@EqualsAndHashCode(callSuper = true)
@PrimaryKeyJoinColumn(name = "pessoa_id")
public class Usuario extends Pessoa {
```

Embora o código fique limpo visualmente, durante a compilação o Java gera dois construtores diferentes. Isso nos permite instanciar um usuário de duas formas

Uma vazia, que o Hibernate usa para carregar dados do banco de dados, e outra completa, que usamos para cadastrar novos usuários.

Polimorfismo

```
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  4 usages  xLua
    Optional<Usuario> findByLogin(String login);  2 usages  xLuanKyrb
}
```

A nossa interface de repositório utiliza **Polimorfismo Paramétrico**. O método .save() ou .findAll() não foi escrito especificamente para a classe Usuario; ele é um método genérico da interface JpaRepository. No entanto, quando o chamamos, ele assume a forma e o comportamento específicos para a nossa entidade Usuario. É o mesmo método (mesma assinatura) a comportar-se de formas diferentes para entidades diferentes.

Hora do Teste Prático

Conclusão e Melhorias

Principais melhorias:

- Adição de alteração de senha no primeiro acesso (Já está no Back falta implentar no sistema)
- Armazenar os dados do Paciente pelos Atendimentos realizados no Protocolo (Apenas armazena local, porém já tem o Back para suportar)
- Dashboard completo de Acompanhamento para controle completo de pacientes e seu estado.
- Dashboard seguindo a LGPD (visto que dados sensiveis não podem ficar expostos em uma tela ‘pública’)
- No cadastro visualizar todos os pacientes existentes, podendo alterar, adicionar sinais vitais, atualizar informações e dar um desfecho ao paciente, vinculando ao atendimento e salvando no histórico do paciente.

Fim